SOP

Classical cryptography relies on the **mathematical hardness** of **factoring** like RSA and ECC. Integer factoring is one of the challenges hidden in the security aspects of crypt analysis. For any protocol, either a **symmetric (same key)** or **asymmetric (public key, private key)** pair is used for encryption and decryption. However, the question is, **how do we create non-deterministic cipher text in every iteration?**The use of **TRNG** instead of **PRNG** is a common practice. **PUF** is one of the techniques to generate **non-deterministic** private keys for **TRNG** applications. Rather than this, **QKD(BB84)** is the best well-known application for **private key generation** shared between two parties using the insecure quantum channel and authenticated on the classical channel. Key distribution can be possible in **two** ways: **based on mathematics (public key cryptography)** or **QKD.** As light has two properties, '**wave property'** and **particle property',** QKD is based on two principles; one is '**Heisenberg uncertainty'** (it is impossible to measure both the 'position' and 'momentum' of an electron at the same time) and the other is **'no-cloning principle'**(it is impossible to create an identical copy of an arbitrary quantum state) which makes QKD quantum safe. Side channel analysis can't break the security wall of QKD due to **entanglement** (can't express one quantum state of a photon particle without other particle quantum states). So, quantum cryptography ensures that communication can't be eavesdropped on without introducing errors (measurement of qubits destroys the quantum state), so QKD can't prevent eavesdropping but must be detected. A qubit (light particle photon) encoded in two bases (rectilinear (horizontal=>0^0 ,vertical=>90^0) and diagonal(45^0 ,135^0) can't be copied without introducing measurement error which invertedly modifies the quantum state of the qubit, which necessary and sufficient condition for identifying the existence of eavesdropping hence quantum safe.BB84 has two parts. The first one is the '**quantum part'**, which uses a quantum channel (critical establishment phase) and the later one is the **'classical part'** (post-processing stage), which uses a classical channel for **sifting,authentication,** and **error estimation(QBER)**, **error detection and correction(error correcting codes)** and **privacy implication**. From the communication channel point of view, the transmission of information between Alice and Bob can occur in various ways. It may be using point-to-point WDM or DWDM, Optical fiber, or some free space. Optical WDM channels are called **'light paths'** and may span multiple fiber links; if there are enough wavelengths for all fiber links, all-optical light paths can connect every node pair, and there is no networking problem to solve. However, the scenario is different; two constraints are considered: 'when a set of light paths are given and need to establish a network'. **Firstly**, the route these light paths should be set up must be determined. **Secondly,** the wavelength assigned to these light paths should be determined to establish a maximum number of light paths. Finally, the goal is to find the shortest route path. A light path operates on the same wavelength across all fiber links it traverses, called the **'wavelength continuity**' constraint, and two light paths that share a common fiber link should not assign the same wavelength. Another interesting scenario is that the light path may switch between different wavelengths on its route, known as the **"RWA"** (routing and wavelength) assignment problem where routing is **'multi-commodity flow'** and wavelength assignment can convert as the '**graph coloring'** problem is **NP-complete**. Still, we don't have any polynomial time solution for this so that we can **minimize** the 'blocking probability' and **maximize** the 'number of connections' known as **DLE**(dynamic light path establishment) called '**safe path**' such that Alice and Bob can share information from both ends with high-speed communication with respect to the presence of tuning parameter **SNR** and '**Bandwidth**' which controls the **QBER** and **secret key rate**. In my future work for performance measurement, we will incorporate "Quantum ML" besides traditional machine learning, which will boost our research goal.

QKD (Prepare & measure ,Quantum Communication & Classical Post Processing)

Assumption and intuition's

BB84 is based on two principles: one is the "No-Cloning theorem", and the other is "Heisenberg Uncertainty" because a "qubit" (light particle=photon) can't be cloned/copied/amplified and measured.

No-Cloning theorem:

One can't clone an unknown quantum state (satisfy uniqueness property) and can't measure without disturbing the system. In other words, a qubit can't be cloned without introducing detectable disturbances, which is called measurement error.

Heisenberg Uncertainty:

Each measurement invertedly modifies the quantum state so that the eavesdropper (Eve) is unable to monitor communication without altering the transmitted information, which can be detected by Alice(sender) and Bob(receiver).

Quantum Communication:

1. Polarized Photons (light particles) can be sent from Alice(sender) to Bob(receiver).
2. The quantum communication phase starts with the generation of a random bit s[illegible]y).
3. Alice encodes the raw key using either rectilinear (horizontal =>90^0 and verti[illegible]gonal ($+45^0$ or -45^0([180^0-45^0]=135^0) basis.
4. Bob decodes the encoded bits by randomly selecting rectilinear or diagonal polarization as a measurement basis.
5. In the case of perfect communication, the bits of Alice and Bob would agree whenever they have chosen identical coding and decoding bases. (Not true in general)

Future Work

EVE task:

Classical Post Processing :

[1]. Sifting: They(Alice & Bob) compare what encoding/decoding bases they used for each bit and keep only those bit values for which bases are matched. This will remove 50% of raw key bits on average.

[2]. Reconciliation or Error correction: This step is used for channel noise removal and ensures an eavesdropper's existence. If QBER is less than an acceptable threshold(<5% =ETSI/CDOT standard), transmission(key distribution) can take place otherwise aborted.

[3]. Privacy Amplification: Scarifying(publicly announcing) some of the key bits after the reconciliation step to exponentially decrease Eve's knowledge of the final key. The assumption is that any channel noise is due to Eve's eavesdropping. In other words, in terms of "Shannon entropy", it is a procedure to minimize Eve's knowledge about the key.

[4]. Authentication: To verify the origin of the correctness of the classical communication message.

Implementation Algorithm:

Quantum Communication: (First part)

1. Alice and Bob communicate over a Quantum Channel. Alice randomly selects a string of bits and a string of bases (rectilinear or diagonal) of equal length.

2. Alice transmits a photon for each bit with the corresponding polarization through an optical fiber (or other channels that allow sending photons) to Bob.

3. Bob randomly chooses a basis for each photon to measure its polarization.

4. If Bob selects the same basis as Alice for a particular photon, he will correctly find the bit Alice wanted to share as he measured the same polarization. If he doesn't guess correctly, he will get a random bit.

Classical Post Processing (Second Part)

5. Bob tells Alice the bases he used to measure each photon. Alice informs Bob of the bases he guessed correctly to measure the encoded bits.

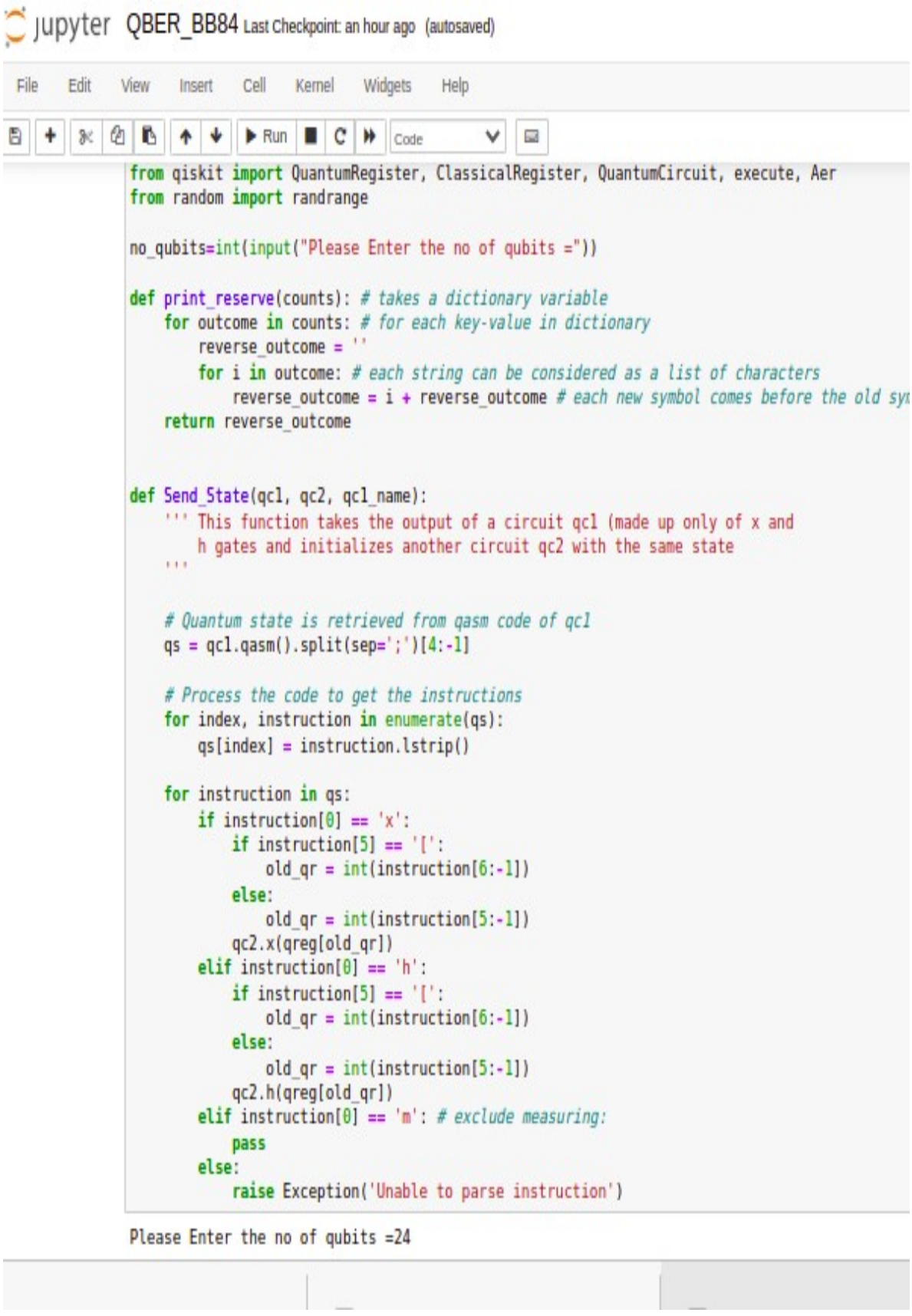
6. Alice and Bob then removed the encoded and measured bits on different bases. Now, Alice and Bob have an identical bit-string, the **shifted key**(reduced bit string due to bases mismatch).

7. To check the presence of Eve, Alice and Bob can share a few bits from the shifted key, which are supposed to be the same. Any disagreement in the compared bits will expose the presence of Eve.

Workout Example:

Quantum Transmission part													
Alice random bit	0	1	1	1	0	1	0	1	1	0	0	1	1
Random bases selected by Alice to send	D	R	D	D	R	D	R	R	D	D	D	R	D
Polarized photons Alice sends (X-basis / Z-basis) X-basis=> Rectilinear Basis, Z-basis=>Diagonal basis	↗	↑	↖	↗	→	↖	→	↑	↖	↗	↗	↑	↖
Random guess of bases by Bob for measurement	D	R	R	D	R	D	D	D	R	D	R	R	D
Bits received by Bob	0	1	0	1	0	1	1	1	0	0	0	1	1
Classical Post-Processing Part													
Bob publicly reveals the bases of received bits that he used for measurement.	D	R	R	D	R	D	D	D	R	D	R	R	D
Alice checks the bases for the match.	√	√	×	√	√	√	×	×	×	√	×	√	√
Shared info without Eve's intervention.	0	1		1	0	1				0		1	1
Bob publicly reveals some key bits. (known to Alice & Eve)		1				1						1	
Alice confirmation		ok				ok						ok	
Final shared secret key (not shared by Bob)	0			1	0					0			1

Output Analysis:



jupyter QBER_BB84 Last Checkpoint: an hour ago (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Code

In [12]:

```
#Step 1: Create Quantum and classical Register of n bits

qreg = QuantumRegister(no_qubits) # quantum register with 4/8/16 qubits
creg = ClassicalRegister(no_qubits) # classical register with 4/8/16 bi

#Step 2: Quantum circuit for Alice state
Alice_QC = QuantumCircuit(qreg, creg, name='Alice')

#Step 3: Declare 3 Array
Alice_send=[] #Initial bit string to send
Alice_basis=[] #Register to save information about encoding basis
Bob_basis=[] #Register to save information about the decoding basis

#Step 4: Creating random bit string
for i in range(no_qubits):
    bit = randrange(2)
    Alice_send.append(bit)

#Step 5:Preparing qubits, apply X gate if the bit is equal to 1
for i, n in enumerate(Alice_send):
    if n==1:
        Alice_QC.x(qreg[i]) # apply x-gate

#Step6: Encoding
for i in range(no_qubits):
    r=randrange(2) #Alice randomly picks a basis
    if r==0: #if the bit is 0, then she encodes in Z basis
        Alice_basis.append('Z')
    else:    #if the bit is 1, then she encodes in X basis
        Alice_QC.h(qreg[i])
        Alice_basis.append('X')

#Step 7: Quantum circuit for Bob's state
Bob_QC = QuantumCircuit(qreg, creg, name='Bob')
Send_State(Alice_QC,Bob_QC, 'Alice') #Alice sends states to Bob
```

QBER_BB84 Last Checkpoint: an hour ago (autosaved)

View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykerne

Run Code

```
#Step 8:Bob measurements of qubits
for i in range(no_qubits):
    r=randrange(2) #Bobs randomly pick a basis
    if r==0: #if the bit is 0, then measures in Z basis
        Bob_QC.measure(qreg[i],creg[i])
        Bob_basis.append('Z')
    else:    #if the bit is 1, then measures in X basis
        Bob_QC.h(qreg[i])
        Bob_QC.measure(qreg[i],creg[i])
        Bob_basis.append('X')

#Step 9: Simulate
job = execute(Bob_QC,Aer.get_backend('qasm_simulator'),shots=1) #Note that Bob only has one shot to measure qubits
counts = job.result().get_counts(Bob_QC) # counts is a dictionary object in python
counts = print_reserve(counts)

#Step 10: Saving Bob received string as a list
Bob_received = list(map(int, counts))

print("Alice sent:", Alice_send)
print("Alice encoding basis:", Alice_basis)
print("Bob received:", Bob_received)
print("Bob decoding basis:", Bob_basis,"\n")
print("Here we don't compare the Basis, hence there may be a chance of eavesdropping")
QBER=0
for i in range(0,len(Alice_send)):
    if(Alice_send[i]==Bob_received[i]):
        QBER=QBER
    else:
        QBER=QBER+1
print("QBER_without_eavasdropping=", QBER)
QBER_percentage=QBER/len(Alice_send)*100
print("QBER_percentage_without_eavasdropping=",QBER_percentage)
QBER=0
print("\n Apply sifting operation by considering basis")
```

```
Alice sent: [1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1]
Alice encoding basis: ['X', 'X', 'X', 'Z', 'X', 'Z', 'X', 'Z', 'X', 'X', 'X', 'X', 'X', 'Z', 'Z', 'Z', 'Z', 'Z',
'X', 'X', 'Z', 'Z', 'X', 'Z']
Bob received: [1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1]
Bob decoding basis: ['X', 'Z', 'X', 'X', 'Z', 'X', 'X', 'Z', 'Z', 'X', 'Z', 'X', 'Z', 'X', 'X', 'Z', 'X', 'X', 'Z',
'Z', 'Z', 'Z', 'Z', 'Z']

Here we don't compare the Basis, hence there may be a chance of eavesdropping
QBER_without_eavasdropping= 5
QBER_percentage_without_eavasdropping= 20.833333333333336

 Apply sifting operation by considering basis
```

jupyter QBER_BB84 Last Checkpoint: 2 hours ago (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted | Python 3 (ipykernel)

Run Code

In [13]:
```
print("Alice sent:", Alice_send)
print("Alice encoding basis:", Alice_basis)
print("Bob received:", Bob_received)
print("Bob decoding basis:", Bob_basis,"\n")

#Sifting
Alice_key=[] #Alice register for matching rounds
Bob_key=[] #Bob registers for matching rounds
for j in range(0,len(Alice_basis)): #Going through list of bases
    if Alice_basis[j] == Bob_basis[j]: #Comparing
        Alice_key.append(Alice_send[j])
        Bob_key.append(Bob_received[j]) #Keeping key bit if bases matched
    else:
        print("Basis mismatch_"+"index =",j,",",Alice_basis[j],Bob_basis[j])
        pass #Discard round if bases mismatched
print("\n")
print("Alice_key_reduced_after_sifting =", Alice_key)
print("Bob_key_reduced_after_sifting =", Bob_key)
```

```
Alice sent: [1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1]
Alice encoding basis: ['X', 'X', 'X', 'Z', 'X', 'Z', 'X', 'Z', 'X', 'X', 'X', 'X', 'X', 'Z', 'Z', 'Z', 'Z', 'Z',
'X', 'X', 'Z', 'Z', 'X', 'Z']
Bob received: [1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1]
Bob decoding basis: ['X', 'Z', 'X', 'X', 'Z', 'X', 'X', 'Z', 'Z', 'X', 'Z', 'X', 'Z', 'X', 'X', 'Z', 'X', 'X',
'Z', 'Z', 'Z', 'Z', 'Z', 'Z']

Basis mismatch_index = 1 , X Z
Basis mismatch_index = 3 , Z X
Basis mismatch_index = 4 , X Z
Basis mismatch_index = 5 , Z X
Basis mismatch_index = 8 , X Z
Basis mismatch_index = 10 , X Z
Basis mismatch_index = 12 , X Z
Basis mismatch_index = 13 , Z X
Basis mismatch_index = 14 , Z X
Basis mismatch_index = 16 , Z X
Basis mismatch_index = 17 , Z X
Basis mismatch_index = 18 , X Z
Basis mismatch_index = 19 , X Z
Basis mismatch_index = 22 , X Z


Alice_key_reduced_after_sifting = [1, 1, 1, 0, 0, 0, 1, 1, 0, 1]
Bob_key_reduced_after_sifting = [1, 1, 1, 0, 0, 0, 1, 1, 0, 1]
```

Maps | Python Senti... | Sentiment An... | Fine-grained S... | A Beginner's... | Shor's Algorit... | ipytho

jupyter QBER_BB84 Last Checkpoint: an hour ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

Run Code

In [14]:
```
print("Alice key reduced after sifting =", Alice_key)
print("Bob key reduced after sifting =", Bob_key)

#QBER with basis matching
rounds = len(Alice_key)//3   #To divide without remainder, use //
no_of_bit_error=0
for k in range(0,rounds):
    random_bit_index = randrange(len(Alice_key))
    tested_reveal_bit_in_public_channel = Alice_key[random_bit_index]
    print ("Alice randomly selected bit index =", random_bit_index, ", and its value is = ",tested_reveal_bit_in_pu
    if(Alice_key[random_bit_index]==Bob_key[random_bit_index]):
        no_of_bit_error=no_of_bit_error
    else:
        no_of_bit_error=no_of_bit_error+1

#removing tested bits from key strings
    del Alice_key[random_bit_index]
    del Bob_key[random_bit_index]
print("\n")
print("QBER=", no_of_bit_error)
QBER_percentage=no_of_bit_error/len(Alice_key)
print("QBER_percentage=",QBER_percentage)
print("Alice's secret key =", Alice_key)
print("Bob' secret key =", Bob_key)
```

```
Alice key reduced after sifting = [1, 1, 1, 0, 0, 0, 1, 1, 0, 1]
Bob key reduced after sifting = [1, 1, 1, 0, 0, 0, 1, 1, 0, 1]
Alice randomly selected bit index = 8 , and its value is =  0
Alice randomly selected bit index = 2 , and its value is =  1
Alice randomly selected bit index = 7 , and its value is =  1


QBER= 0
QBER_percentage= 0.0
Alice's secret key = [1, 1, 0, 0, 0, 1, 1]
Bob' secret key = [1, 1, 0, 0, 0, 1, 1]
```

In [ ]:

Challenges of BB84

1. No clear guidelines we found literature's for understanding the correlations between ‘QSNR’ and ‘QBER’ with the varying distance. As the the distance increases how SNR decreases which may lead to a high QBER.

2. How dark count rate can have an impact on QBER.

3. BB84 ensure the tracking of eavesdropping but can’t resist ,can we do better?

4. As per C-DOT QBER<5% for maintaining communication, how to ensure it without aborting any transmission in between.

5. We don’t have any physical hardware setup for BB84(QKD) implementation . In real scenario how light particle encodes classical bits in quantum bits and transmit with the presence of noise inside the channel which have a high capability of bit flipping such that we can find out the QBER in real time for 1000 of iterations ,when the same message can transmit with varying distance.

6. After ‘Shifting’ stage in BB84 approx 50% bits are wasted due Alice and Bob’s bases mismatches ,hence the size of raw key is reduced ,so full bit lengths are not used.

7. To reduce the knowledge of Eve about the ‘Key’ again some of the bits publicly announced by Bob for cross verification form Alice end ,but unfortunately Eve also know the sacrificed bit-string ,so again final key size is reduced. We can’t able to use the full length bit stream as the final key.

Future Plan

1. Can we able to use full bit string for reliable communication by correcting errors (using LDPC,Convolution codes or any ANN based solution.

2. Try to figure out the correlation among Distance ,QBER and (SNR/ OSNR) to estimate the secret key rate .

3. Try to make a corpus for QKD Keys as a demand and supply management platform for secure communication(quantum safe).

4. Study and try to apply different types of attack to check the vulnerability of the protocol.

5. Although we are dealing with single photon encoding scheme , if we consider multi photon encoding scheme for better encoding but how to resist PNS (Photon number splitting) attack.